

Multilevel Hypergraph Partitioning: Applications in VLSI Domain

论文总览：问题、主线与贡献

论文：George Karypis, Rajat Aggarwal, Vipin Kumar, Shashi Shekhar
IEEE TVLSI, 1999

学习日期：2026 年 9 月 15 日

今日目标

今天不追求记住全部算法细节，而要能在不看论文的情况下回答四个问题：

1. 什么是超图二分，它为什么适合表示 VLSI 电路？
2. hMETIS 的“粗化—初始划分—反粗化/细化”三阶段分别做什么？
3. 论文真正的新意为什么主要在**直接作用于超图的粗化**？
4. 作者用什么实验事实支持“质量更好且速度更快”？

建议学习时长：约 90 分钟。本文是第一天的导学讲义兼复习笔记。

目录

1 先用一分钟抓住整篇论文	3
1.1 论文的输入、约束与输出	3
1.2 最重要的因果链	4
2 论文全景地图	4
2.1 章节与问题对应关系	4
2.2 三阶段框架	5
3 核心方法：hMETIS 每一阶段做什么	6
3.1 粗化：EC、HEC 与 MHEC	6
3.2 最粗层：不要迷信“最小 coarse cut”	7
3.3 反粗化：FM、早停 FM 与 HER	7
3.4 受限粗化与多阶段细化	7
4 实验怎样支持论文主张	7
4.1 实验协议先于数字	7
4.2 应记住的结果量级	8

5	贡献、假设与局限	8
5.1	贡献清单	8
5.2	隐含假设与阅读时的警惕	9

1 先用一分钟抓住整篇论文

一句话摘要

hMETIS 不把超图粗暴地转换成普通图，而是直接在超图上反复**缩小问题、求一个全局粗解、逐层放大并局部修正**，从而在 VLSI 电路二分任务上同时获得较小的超边割和较短的运行时间。

1.1 论文的输入、约束与输出

设超图为

$$H = (V, E), \quad e \in E \text{ 可以同时连接两个以上的顶点.}$$

VLSI 网表中的对应关系是：

$$\boxed{\text{cell / module} \longleftrightarrow \text{vertex}}, \quad \boxed{\text{net} \longleftrightarrow \text{hyperedge}}.$$

目标是把 V 分成 V_0, V_1 ，同时满足：

$$V_0 \cap V_1 = \emptyset, \quad V_0 \cup V_1 = V,$$

$$\text{balance}(V_0, V_1) \text{ 满足给定平衡约束,}$$

$$\min \text{ cut}(V_0, V_1) = |\{e \in E : e \cap V_0 \neq \emptyset, e \cap V_1 \neq \emptyset\}|.$$

也就是说，一条网只要跨越两个分区，就计为一条被切超边；它连接三个还是十个单元，并不会自然等价于若干条普通边的割代价。

为什么不能简单“超边变团 (clique)”？

把大小为 k 的超边替换成一个 k 点团，会改变目标函数：普通图中同一条网跨分区时可能产生多条割边，而原超图目标只把它算作一条被切网。论文强调，不存在一种团内统一赋权方式能在所有切分下精确保持原超边割代价；转换还会破坏稀疏性。因此 hMETIS 选择从粗化到细化都直接处理超边。

1.2 最重要的因果链



图 1: hMETIS 的纵向核心工作流：先在粗层把握全局结构，再逐层恢复自由度并修正局部边界

论文结论部分把这一点概括为：**粗化提供 global view，逐层细化提供 local view。**

2 论文全景地图

2.1 章节与问题对应关系

章节	回答的问题	第一天掌握程度
I 引言	为什么要做超图划分？既有 KL/FM、两阶段和多层方法缺什么？	能复述动机与缺口
II 多层二分	三阶段怎样工作？EC、HEC、MHEC、FM、HER 各是什么？	看懂框架和方法角色
III 多阶段 细化	已有划分怎样借助受限粗化继续改进？	知道核心思想即可
IV 实验	与 PROP、GMetis、PARABOLI 等相比如何？	记住协议和量级
V 结论	为什么有效？扩展性来自哪里？	能画出因果链

2.2 三阶段框架

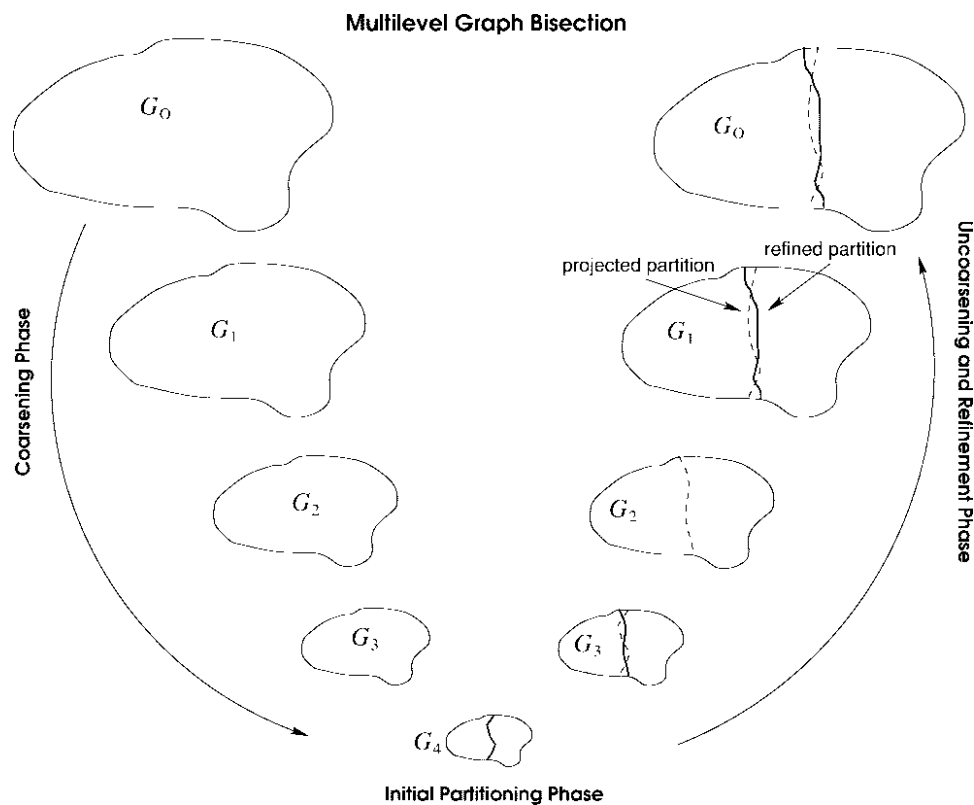


图 2: 多层二分的三个阶段: 粗化、最粗层初始划分、反粗化与逐层细化 (论文 Fig. 1) (原论文图, 供学习引用)

把图中的流程读成一个算法:

1. **Coarsening (粗化)**: 构造 $H_0, H_1, \dots, H_m$, 其中 H_0 是原超图, 规模逐层减小; 彼此关系紧密的顶点被聚合为粗顶点。
2. **Initial partitioning (初始划分)**: 在通常少于 200 个顶点的 H_m 上求平衡二分, 再用 FM 改进。作者不追求最粗层的精确最优, 因为后续仍会显著修改。
3. **Uncoarsening/refinement (反粗化/细化)**: 把粗层划分投影到下一细层, 并用 FM 或 HER 降低 cut; 反复进行直到 H_0 。

为何“逐层”比只在原图上跑 FM 更强?

粗层移动一个聚合顶点, 相当于在原图中同时移动一组相关顶点, 因而能进行大尺度搜索; 投影后的划分又是下一层 FM 的高质量起点。算法于是从“大步调整全局结构”平滑过渡到“小步打磨局部边界”。

3 核心方法：hMETIS 每一阶段做什么

3.1 粗化：EC、HEC 与 MHEC

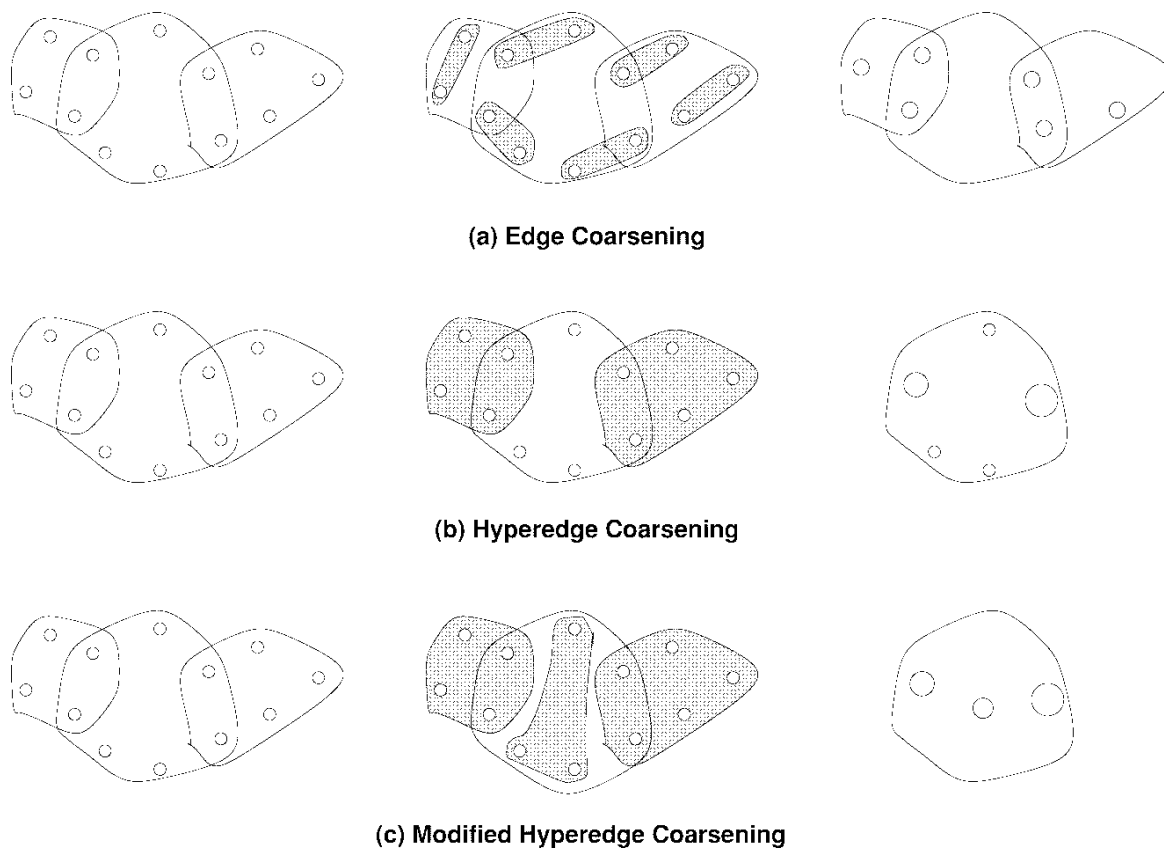


图 3: 三类顶点聚合方式：成对聚合、整条超边聚合、以及改进的超边聚合（论文 Fig. 2）（原论文图，供学习引用）

EC：边粗化

随机访问顶点，为每个未匹配顶点选择一个与它共享超边、且连接权重最大的未匹配邻点，形成二元匹配。它类似隐式使用 clique 表示，但不真的生成稠密普通图。

HEC：超边粗化

选择相互独立的超边，把每条被选超边上的全部顶点收缩在一起。优先高权重、较小的超边，目的是更快减少粗层中仍“暴露”的超边权重。

MHEC：改进超边粗化

先做 HEC，再第二遍处理未被收缩的超边，把其中仍未归组的顶点进一步聚合，以缓解 HEC 中许多超边缩得不够、粗顶点权重差异过大的问题。

第一天需要形成的判断

这篇论文不是简单地“把 FM 多跑几遍”。它最关键的工程与算法判断是：**粗化质量决定多层框架能否保留原问题的结构**。强粗化让最粗层的解仍有意义，也使后续只需较轻量的细化。

3.2 最粗层：不要迷信“最小 coarse cut”

作者使用随机平衡二分或 BFS 区域生长，再以 FM 细化。随机性产生多个候选；实验配置在最粗层生成 10 个初始划分，并在反粗化中淘汰 cut 比当前最佳差 10% 以上者。论文报告：保留 10 个候选平均可再降 cut 约 3%–4%，而耗时约增加一倍；继续增加到 20 个，平均额外改善不足 0.5%。

这里的深层含义是：最粗层上的 cut 只是原问题质量的**代理指标**。某个当前 cut 稍高的候选，在恢复自由度并细化后反而可能更好，因此需要保留有限的“搜索多样性”。

3.3 反粗化：FM、早停 FM 与 HER

- **FM**：按移动顶点带来的 gain 选择动作，并满足平衡约束；一个 pass 中顶点移动后锁定，最后回滚到该 pass 所见的最小 cut 状态。
- **FM-EE (early exit)**：多层投影已经给出很好的起点，因此最多做两个 pass；若连续进行了约当前顶点数 1% 的、未改善 cut 的移动，就提前结束该 pass。目标是以很少质量损失换取显著加速。
- **HER (hyperedge refinement)**：针对一条跨区超边，考虑把它在某一侧的整组顶点移到另一侧，从而一次消除整条 cut hyperedge；它弥补单顶点 FM 不擅长处理“两侧都有很多顶点”的大超边这一弱点。

3.4 受限粗化与多阶段细化

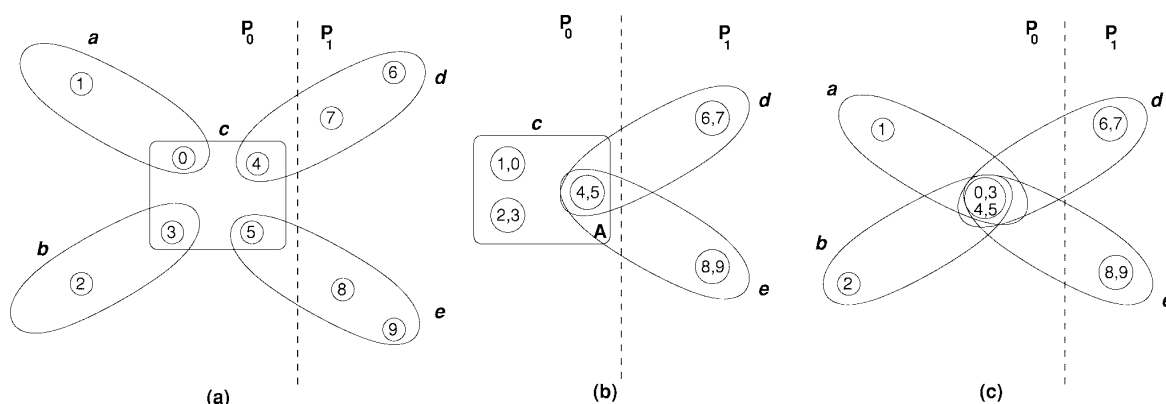


图 4: 同一划分经不同受限粗化会暴露不同的改进机会 (论文 Fig. 3) (原论文图, 供学习引用)

给定已有划分, **restricted coarsening** 只允许合并同一分区内的顶点, 因此粗化前后的 cut 数保持不变。随后用不同随机粗化结构进行反粗化, 就可能逃离原来的局部最优。这个过程可重复, 并形成论文所讨论的多阶段 / 循环细化策略。

直觉类比：第一次把山地地图折叠的方式让你走进了一个谷底；保持当前海拔不变，换一种折叠方式再展开，可能出现此前看不见的下降路径。

4 实验怎样支持论文主张

4.1 实验协议先于数字

- 数据：ACM/SIGDA 电路划分 benchmark, 共 23 个电路实例；

- 平衡要求：45–55；
- 指标：被切超边数（越小越好）与运行时间；
- 平台：SGI Challenge, 200 MHz MIPS R10000；跨论文时间比较需考虑机器差异；
- 随机算法通常报告多次运行中的最好结果，具体变体所用次数在 Table II 中说明。

4.2 应记住的结果量级

- 摘要的总结：相对其他当时先进方法，平均 cut 改善约 **6%–23%**；运行时间常减少到对方的约 1/4–1/10。
- 规模超过 100,000 顶点的超图，在当时工作站上可于数分钟内二分。
- 对较大实例（至少 10K 顶点），相对优势更稳定、更明显；论文给出的 cut 改善范围约为 **9%–30%**。
- 与先把超图近似成图的 GMetis 相比，hMETIS 直接优化真实的 hyperedge cut；在最大 benchmark `golem3` 上，GMetis 的 100 次运行最佳结果仍比 hMETIS-FM 的 10 次运行结果差 50%。

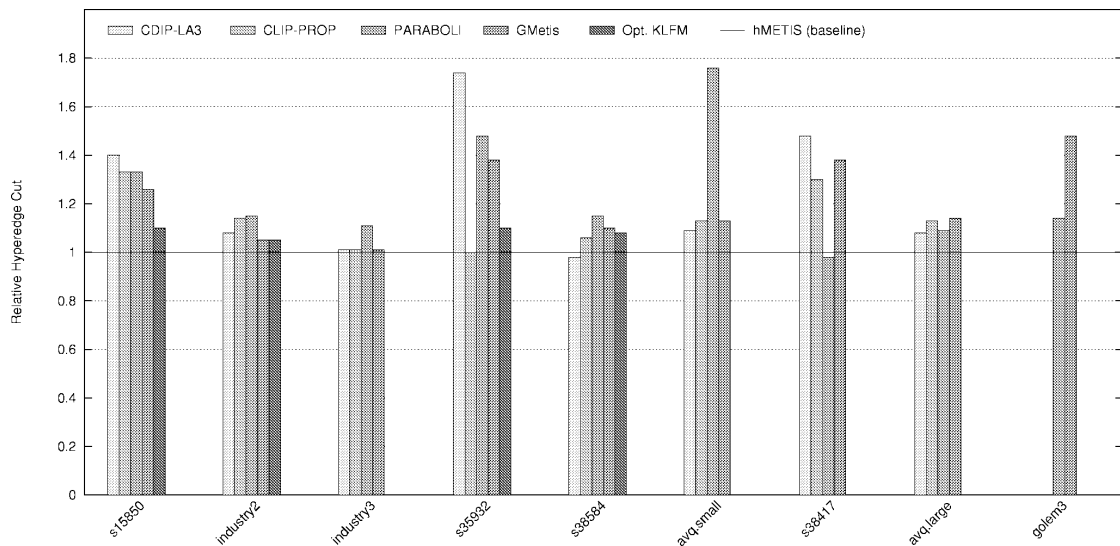


图 5: hMETIS 在大规模 benchmark 上相对既有方法的表现（论文 Fig. 4）（原论文图，供学习引用）

如何批判性阅读这些结果？

“更快”并非完全同机同实现比较，作者对不同 CPU 做了估算缩放；“最好结果”也依赖重复运行预算。因此最稳健的结论是：在论文给定 benchmark、平衡约束和运行协议下，hMETIS 显示出明显的质量优势与良好扩展趋势，而不是无条件地对所有超图都占优。

5 贡献、假设与局限

5.1 贡献清单

1. 提出直接作用于超图的多层划分框架 hMETIS，避免 clique 转换的目标失真与稠密化。

2. 提出并比较 EC、HEC、MHEC 等粗化策略，尤其强调强粗化对最终质量的决定作用。
3. 在高质量投影解的前提下简化 FM（包括早停），取得质量与速度平衡。
4. 引入 HER，以及基于受限粗化的多阶段细化思路，让已有划分可通过不同层次结构继续改进。
5. 在 23 个 VLSI benchmark 上展示当时领先的 cut 质量与较好扩展性。

5.2 隐含假设与阅读时的警惕

- 主要目标是平衡二分下的 cut-net 数；现实设计还可能同时关心时序、拥塞、面积、I/O 上限和多约束权重。
- 随机粗化和多次运行是质量来源之一；比较算法时应统一总计算预算，而不只统一“运行次数”。
- 1999 年的 benchmark 与硬件不足以直接代表现代超大规模网表，但多层思想及直接保持超图语义的原则仍具基础价值。
- 论文强调经验结果和算法设计，未给出最优性保证；它是高效启发式算法，而非精确算法。

明天建议 第二天进入“算法细读”：用一个手工超图逐步模拟一次 HEC/MHEC 粗化、一次投影和一个 FM pass，并明确 gain 更新与平衡约束。

资料来源与说明

本文只依据用户提供的论文：G. Karypis, R. Aggarwal, V. Kumar, and S. Shekhar, “Multilevel Hypergraph Partitioning: Applications in VLSI Domain,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 7, no. 1, pp. 69–79, Mar. 1999。文中四幅参考图分别摘自原论文 Fig. 1–4，仅用于本次个人学习笔记；图题已明确标注来源。实验数字均按原论文摘要、实验与结论部分整理。